- `{{ form.as_table }}` will render them as table cells wrapped in <tr> tags.
- `{{ form.as_p }}` will render the fields wrapped in <p> tags.
- `{{ form.as_ul }}` will render them wrapped in <li> tags.

However, sometimes, developers need to manually render the form to be able to customise the user interface.

Form Validation

In this section, we will discuss an important part of the Django forms – Form Validation. As you know, while filling out a form, there is a chance of junk data entering the system. Django provides a few built-in methods for the auto-validation of the data entered through the forms. Using these methods, you can restrict the submission of forms without entering the correct data. This is a clean and easy approach for data validation. Django forms can be submitted only with the CSRF tokens.

The data validation is done with the is_valid() method. The method gives a boolean value as an output. If the output returns True, this means that the input data is valid. This valid data is then placed in a cleaned_data attribute.

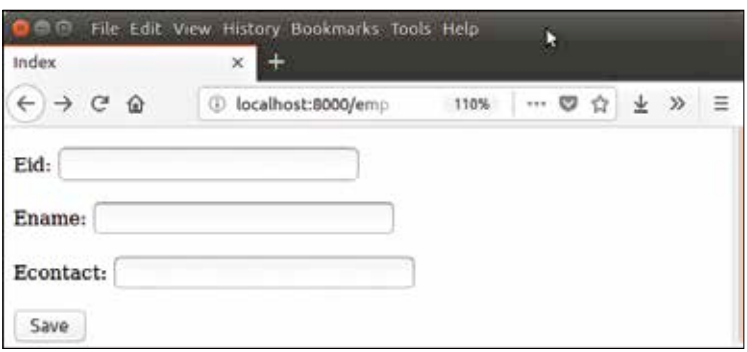

Image source: https://www.javatpoint.com

Let us try and recreate this form. We will also add the data validation using the is_valid() method to make sure that the correct data enters the database. The code for the employee form as shown in the image is attached below.

First, we need to create a model that contains the fields as well as their data types.

```
from django.db import models
class Employee(models.Model):
    eid = models.CharField(max _ length=20)
```